zkDietFormer

Anonymous Author(s)

Abstract

Keywords

Zero-Knowledge Proofs

1 Introduction

Recent breakthroughs in Transformer-based models have revolutionized artificial intelligence, facilitating the widespread deployment of high-performance Large Language Models (LLMs) via Model-as-a-Service (MaaS) platforms. In this setting, model providers typically large models, while users receive outputs through APIs or remote inference interfaces. However, this architecture inherently obscures the generation process: users cannot independently verify whether the received output truly produced from the specific model architecture and weight parameters intended. Users are therefore forced to rely on trust regarding whether the provider substituted a cheaper model for the same input, post-processed or modified outputs due to internal policies, or unintentionally changed the model during updates or deployment. This problem of model execution integrity is particularly critical in high-stakes domains such as finance, healthcare, and security, and becomes increasingly central as demands for transparency and accountability in inference continue to grow.

One may attempt to address this trust issue by requiring the model provider to disclose model parameters for verification. In practice, however, this is practically infeasible. Model weights are the result of substantial investments in data, compute, and engineering, and are protected as proprietary intellectual property (IP). Consequently, a fundamental dilemma arises: users demand integrity guarantees for the model's outputs, while providers must preserve the privacy of their model parameters to protect their assets.

Zero-knowledge proofs (ZKPs) provide a principled way to reconcile these conflicting requirements. By leveraging ZKPs, a prover (the model provider) can cryptographically prove that an output y is the result of evaluating the model on the user's input x with a secret weight W, i.e., $f(x, W) = y$, without revealing any information about W itself. This enables verifiable inference while maintaining the confidentiality of model parameters.

However, making ZKP-based verifiable AI practically deployable requires the efficient proving of computationally intensive deep learning operations. In modern Transformer architectures, the primary bottleneck is the self-attention mechanism, the computational core of the model. For an input sequence of length n and hidden dimension d, the input $X \in \mathbb{F}^{n\times d}$ is projected into queries, keys, and values $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{F}^{n\times d}$. Standard self-attention forms an attention matrix $\mathbf{QK}^\top \in \mathbb{F}^{n\times n}$, applies a row-wise softmax over the entire matrix, and then multiplies by $\mathbf{V}$ to obtain the attention output.

This computation requires $O(n^2 d)$ linear operations for matrix multiplications, as well as an additional $O(n^2)$ nonlinear cost induced by the softmax. The resulting computational overhead is substantial, posing the single largest obstacle to proving large-scale Transformer models.

The most naïve approach to proving self-attention involves arithmetizing the full $\mathbf{QK}^\top$ computation into a circuit and enforcing the resulting $O(n^2 d)$ operations within a SNARK. However, this cost far exceeds practical limits. To mitigate this, many systems have adopted probabilistic verification techniques for large matrix products, such as Freivalds' algorithm [11] or Thaler's randomized checks [23]. These methods verify the $n \times n$ multiplication relation without full recomputation by sampling random vectors, compressing verification to a small number of checks and reducing the cost to $O(n^2)$. Nevertheless, the complexity remains dominated by the quadratic term, which becomes increasingly prohibitive as models scale. For instance, while BERT [9] uses 512 tokens and GPT-3 [2] uses a 2,048-token context window, frontier models now support contexts in the hundreds of thousands or even millions (e.g., 400K for GPT-5 [19] and 10M for Llama 4 Scout [18]). As n grows, this quadratic verification cost renders proof generation for realistic sequence lengths impractical.

Meanwhile, deep learning literature has repeatedly demonstrated that attention matrices often exhibit an approximate low-rank structure. Concretely, when $k \ll n$, the attention matrix can be effectively approximated by a rank-k representation, enabling training and inference costs to approach linearity with respect to the sequence length [25, 29]. Despite these advances, the field of Verifiable AI has yet to exploit low-rank structures as a primary mechanism for reducing proof generation costs. Consequently, a gap persists: while low-rank approximations improve computational efficiency in practice, the lack of proof systems tailored to these structures keeps proof complexity anchored at $O(n^2)$. This work aims to bridge this gap. Our central idea is to combine low-rank structures with probabilistic verification to eliminate the quadratic complexity in verifying self-attention matrices.

1.1 Our Contribution

In this work, we propose **zkDietFormer**, a verification protocol that eliminates the dominant $O(n^2)$ bottleneck in proving self-attention by combining a low-rank representation of the attention matrix, Freivalds' randomized checks, and a commit-and-prove SNARK. Our main contributions are summarized as follows:

- **The first verification framework tailored to low-rank self-attention.** To the best of our knowledge, we present the first verification framework specifically designed for

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Proceedings on Privacy Enhancing Technologies YYYY(X), 1–15

© YYYY Copyright held by the owner/author(s).

https://doi.org/XXXXXXX.XXXXXXX

low-rank self-attention. We instantiate this framework using the Nyström method, reducing the overall circuit complexity—including both the core matrix operations of self-attention and the nonlinear operations induced by row-wise softmax—to $O(n)$.

- **Constraint-efficient verification via Freivalds' checks and CP-SNARKs.** We employ Freivalds' algorithm to efficiently verify large matrix computations inside the circuit, while leveraging commit-and-prove SNARKs to offload computationally expensive operations outside the circuit, significantly reducing the total number of constraints.
- **Practical implementation and evaluation.** We implemented a prototype of the proposed framework and evaluated its performance through benchmarks. The results demonstrate a significant reduction in prover costs for practical sequence lengths, showing that **zkDietFormer**is not only theoretically efficient but also practically deployable.

In summary, the prover's constraint cost in **zkDietFormer**scales linearly with the input sequence length n. Consequently, to the best of our knowledge, this work provides the first verifiable mechanism for low-rank self-attention that enables proof generation with cost linear to the sequence length.

2 Related Works

In this section, we provide a brief overview of the core operations of Transformer models and review prior work aimed at improving the efficiency of matrix multiplication, which is widely recognized as a major computational bottleneck. We also examine existing approaches for constructing zero-knowledge proofs for matrix multiplication.

Low-Rank Transformer The Transformer is a deep learning architecture proposed by Google in 2017 [24], which leverages the attention mechanism as its core component to effectively model relationships within sequential data, such as words in a sentence. The core computation of this model consists of matrix multiplications among the query, key, and value matrices. In standard Transformers, these operations incur quadratic computational complexity with respect to the input sequence length n, causing the computational cost to grow rapidly as n increases. To mitigate this bottleneck, Child et al. [6] proposed the Sparse Transformer, which restricts attention to predefined sparse patterns, using strided attention for data with periodic structures such as images and audio, and fixed attention for text without structure. However, this approach relies on implementations tied to specific TensorFlow versions. To address this limitation, Longformer [1] introduced sliding-window attention, where each token attends only to nearby tokens, providing a more general and maintainable solution. Rae et al. [20] proposed the Compressive Transformer, which compresses past hidden activations(memories) using pooling operations and jointly attends to both recent memories and compressed long-term memories to efficiently model long-range context. Nevertheless, these approaches do not preserve full attention, leading to limitations such as insufficient modeling of long-range dependencies [1, 6], or performance degradation in exchange for computational efficiency [1]. Accordingly, Linformer [25] shows that the attention matrix induced by self-attention exhibits a low-rank structure both theoretically and empirically, and approximates scaled dot-product attention by projecting the key and value matrices into a lower-dimensional space via linear projections. As a result, the time and space complexity of self-attention are reduced to linear in the sequence length n. However, Linformer has limitations in causal modeling, where future tokens must be masked, and can incur large mean squared errors compared to exact attention. To overcome these issues, Performer [7] introduces a linear-complexity attention mechanism that avoids explicit projection matrices and low-rank assumptions, and instead approximates full softmax attention via FAVOR+, which is based on positive orthogonal random features for softmax kernel estimation. Nyströmformer [29] introduces a Nyström low-rank approximation that reconstructs the full attention map using a subset of landmark tokens, achieving higher accuracy than Performer while maintaining linear complexity with respect to the sequence length.

zk-SNARKs for Matrix Multiplication There has been ongoing research on verifying the integrity of matrix multiplication using zero-knowledge proofs. Since convolution operations can be reduced to structured matrix multiplication, such verification techniques have been extensively studied as a core technique for proving the correctness and integrity of AI model execution.

Thaler et al. [23] introduced the reuse-of-work technique into the Sumcheck protocol for matrix multiplication verification, reducing the prover's computational overhead from $O(n^3)$ to $O(n^2)$ for $n \times n$ matrices. zkCNN [17] adopts this optimized Sumcheck protocol to efficiently verify matrix multiplication in fully connected layers. pvCNN [26] transforms convolution operations, which inherently require a large number of multiplication gates, into a single matrix multiplication by reshaping the filters and inputs into rectangular matrices. Inspired by the QPP (Quadratic Polynomial Programs) approach of vCNN [16], which applies polynomial representations within the QAP framework, pvCNN further extends this idea by introducing the QMP (Quadratic Matrix Program). As a result, the convolution can be expressed and verified using only a single multiplication gate. zkML [5] applies Freivalds' algorithm [11] to verify matrix multiplication operations in linear layers, such as convolutions and batched matrix multiplications, achieving verification complexity of $O(n^2)$. zkLLM [22] leverages the dedicated sumcheck protocol proposed in SafetyNets [12] to verify matrix multiplication operations arising in Transformer models. As a result, the prover's computational complexity scales linearly with the sizes of the input matrices, achieving a complexity of $O(mn + np)$. zkMatrix [8] reduces matrix multiplication to four inner-product relations and applies Bulletproofs-based verification acceleration techniques, thereby achieving a prover time of $O(n^2)$ and a verifier time of $O(\log n)$ for $n \times n$ committed matrix multiplication. zkVC [30] introduces Constraint-reduced Polynomial Circuit(CRPC) and the Prefix-Sum Query(PSQ) technique to significantly reduce the constraint complexity of matrix multiplication from $O(n^3)$ to $O(n)$.

However, these approaches still require a prover computation complexity of $O(n^2)$ with respect to the matrix dimension (or sequence length) [5, 8, 16, 17, 23], or, even when achieving linear-time complexity in theory, suffer from the limitation that the other matrix dimension m remains prohibitively large in practice [22]. Moreover, nonlinear functions such as softmax cannot be directly handled within ZKP circuits and thus must be approximated, which leads to

accuracy degradation and necessitates the design of model architectures specifically tailored for zero-knowledge proof systems [30].

3 Preliminaries

3.1 Scaled Dot-Product Attention

The model represents interactions between tokens using queries and keys of dimension d_k, and values of dimension d_v. For each token, its associated query computes its similarity scores with the keys of all other tokens in the sequence, thereby determining which parts of the input the model should attend to more strongly. The contextual information is then aggregated by computing a weighted sum of the corresponding value vectors based on these similarities.

The attention mechanism is defined as a function that takes a single query together with a set of key–value pairs and produces an output vector, where the query, keys, values, and outputs all lie in vector spaces. In practice, multiple queries are processed simultaneously by stacking them into matrices $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$. These matrices are obtained by projecting the input matrix $\mathbf{X}$ using learnable parameter matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, and $\mathbf{W}^V$, such that $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}^K$, and $\mathbf{V} = \mathbf{X}\mathbf{W}^V$. Concretely, the attention weights are obtained by scaling the dot product between queries and keys by $1/\sqrt{d_k}$, followed by the application of the softmax function. The final output is computed as the weighted sum of the value vectors using these attention weights.

The output matrix is computed as follows:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

3.2 Multihead Attention

Multi-head attention replaces a single attention function over d_{model}-dimensional queries, keys, and values with h parallel attention heads. The queries, keys, and values are first linearly projected h times into lower-dimensional subspaces of sizes d_q, d_k, and d_v, respectively. For each head i,

$$\text{head}_i = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)$$

where $\mathbf{Q}_i \in \mathbb{R}^{n\times d_q}, \mathbf{K}_i \in \mathbb{R}^{n\times d_k}$ and $\mathbf{V}_i \in \mathbb{R}^{n\times d_v}$. The resulting head outputs are concatenated and linearly transformed to form the final representation:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O$$

with $\mathbf{W}^O \in \mathbb{R}^{hd_v\times d_{\text{model}}}$. This design enables the model to attend to diverse representation subspaces while maintaining computational cost comparable to single-head attention through dimensionality reduction within each head.

3.3 Nystrom-Based Linear Attention

Nyströmformer applies the Nyström method []—a technique widely used in matrix approximation—to reduce the computational complexity of the standard Transformer from $O(n^2)$ to $O(n)$. The key idea of the Nyström method is to reconstruct the full matrix approximately by using only a subset of its rows and columns, instead of computing the entire matrix explicitly.

When the softmax matrix S is expressed as a block matrix as shown below,

$$S = \text{softmax}(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}) = \begin{bmatrix} A_s & B_s \\ F_s & C_s \end{bmatrix}$$

the Nystrom method approximates the expensive block $C_S \in \mathbb{R}^{(n-m)\times(n-m)}$ by using the smaller blocks $A_S \in \mathbb{R}^{m\times m}, B_S \in \mathbb{R}^{m\times(n-m)}, F_S \in \mathbb{R}^{(n-m)\times m}$ as illustrated below:

$$\hat{S} = \begin{bmatrix} A_S & B_S \\ F_S & F_S A_S^+ B_S \end{bmatrix} = \begin{bmatrix} A_S \\ F_S \end{bmatrix} A_S^+ \begin{bmatrix} A_S & B_S \end{bmatrix}$$

Here, A_S^+ denotes the Moore-Penrose pseudoinverse of A_S.

However, when this approach is applied directly to self-attention, the row-wise normalization property of the softmax function still requires accessing all entries of the $n\times n$ matrix in order to compute the denominators. To address this issue, Nyströmformer partitions the entire sequence into m segments before applying softmax, and computes the mean within each segment to obtain m landmark points for the queries and keys, respectively. These landmarks are then used to construct the $n \times m$ matrix $\tilde{\mathbf{Q}}$ and the $m \times n$ matrix $\tilde{\mathbf{K}}$, respectively. Using these matrices, Nyströmformer derives a final approximation that achieves linear computational complexity.

$$\hat{S}V = \text{softmax}\left(\frac{\mathbf{Q}\tilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)\text{softmax}\left(\frac{\tilde{\mathbf{Q}}\tilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)^+\text{softmax}\left(\frac{\tilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right)V$$

The exact computation of the pseudoinverse A_S^+ (e.g., via SVD) is computationally expensive and poorly suited for GPU execution, becoming a practical bottleneck in implementation. To mitigate this issue, Nyströmformer introduces an iterative algorithm that approximates the pseudoinverse using repeated matrix multiplications. The resulting approximation Z^* is then substituted into the final formulation, significantly improving computational efficiency.

$$\hat{S}V = \text{softmax}\left(\frac{\mathbf{Q}\tilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)Z^*\,\text{softmax}\left(\frac{\tilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right)V$$

3.4 zk-SNARKs

A *zero-knowledge succinct non-interactive argument of knowledge*(zk-SNARKs) for an NP relation $\mathcal{R}$ consists of three PPT algorithms Π = (Setup, Prove, Verify):

- $(\text{pk}, \text{vk}) \leftarrow \Pi.\text{Setup}(1^\lambda, \mathcal{R})$: On input the security parameter λ and a relation $\mathcal{R}$, the setup algorithm outputs a proving key pk and a verification key vk.
- $\pi \leftarrow \Pi.\text{Prove}(\text{pk}, \mathbf{x}; \mathbf{w})$: On input the proving key pk, a statement $\mathbf{x}$, and a witness $\mathbf{w}$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, the prover outputs a proof π.
- $0/1 \leftarrow \Pi.\text{Verify}(\text{vk}, \mathbf{x}, \pi)$: On input the verification key vk, a statement $\mathbf{x}$, and a proof π, the verifier outputs 0(reject) or 1(accept)

A zk-SNARK system Π satisfies the following properties:
Completeness. For every true statement for the relation $\mathcal{R}$, an honest prover holding a valid witness always convinces the verifier, i.e., $\forall(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$:

$$\Pr\left[\begin{matrix}(\text{pk}, \text{vk}) \leftarrow \Pi.\text{Setup}(1^\lambda, \mathcal{R}) \\ \pi \leftarrow \Pi.\text{Prove}(\text{pk}, \mathbf{x}; \mathbf{w})\end{matrix} : \quad \Pi.\text{Verify}(\text{vk}, \mathbf{x}, \pi) = 1\right] = 1$$

Knowledge Soundness. Knowledge soundness requires that any prover producing a proof π that makes the verifier accept must possess a corresponding witness, and that a knowledge extractor $\mathcal{E}$ can efficiently extract such a witness from π. Formally, for any PPT adversary $\mathcal{A}$, the following is *negligible*.

$$\Pr\left[\begin{array}{lll} (\mathsf{pk},\mathsf{vk}) \leftarrow \Pi.\mathsf{Setup}(\mathcal{R}) & & \Pi.\mathsf{Verify}(\mathsf{vk},\mathbf{x}^*,\pi^*)=1 \\ (\mathbf{x}^*,\pi^*) \leftarrow \mathcal{A}(\mathsf{pk},\mathsf{vk}) & : & \wedge \\ \mathbf{w} \leftarrow \mathcal{E}_{\mathcal{A}}(trans_{\mathcal{A}}) & & (\mathbf{x}^*;\mathbf{w}) \notin \mathcal{R} \end{array}\right]$$

Zero-Knowledge. There exists a PPT simulator $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ such that $\mathcal{S}_1$ outputs simulated setup parameters $(\mathsf{pk},\mathsf{vk})$ together with a trapdoor td. On input $(\mathsf{pk},\mathsf{vk},td)$, $\mathcal{S}_2$ outputs a simulated proof π. Moreover, for all PPT adversaries $\mathcal{A} = (\mathcal{A}_1,\mathcal{A}_2)$, we have that

$$\left|\Pr\left[\begin{array}{ll} (\mathsf{pk},\mathsf{vk}) \leftarrow \Pi.\mathsf{Setup}(1^\lambda,\mathcal{R}) \\ \mathcal{A}_2(\mathsf{vk},\mathbf{x},\pi)=1 : & (\mathbf{x},\mathbf{w}) \leftarrow \mathcal{A}_1(1^\lambda,\mathsf{pk},\mathsf{vk}) \\ & \pi \leftarrow \Pi.\mathsf{Prove}(\mathsf{pk},\mathbf{x};\mathbf{w}) \end{array}\right]\right.$$
$$\left.-\Pr\left[\begin{array}{ll} (\mathsf{pk},\mathsf{vk},td) \leftarrow \mathcal{S}_1(1^\lambda,\mathcal{R}) \\ \mathcal{A}_2(\mathsf{vk},\mathbf{x},\pi)=1 : & (\mathbf{x},\mathbf{w}) \leftarrow \mathcal{A}_1(1^\lambda,\mathsf{pk},\mathsf{vk}) \\ & \pi \leftarrow \mathcal{S}_2(\mathsf{pk},\mathbf{x},td) \end{array}\right]\right| \le \mathsf{negl}(\lambda).$$

Succinctness. For any statement $\mathbf{x}$ and witness $\mathbf{w}$, the size of proof π satisfies $|\pi| = poly(\lambda)\cdot polylog(|\mathbf{x}|+|\mathbf{w}|)$

Commit-and-Prove SNARKs (CP-SNARKs). A *Commit-and-Prove SNARKs* (CP-SNARKs) [3, 4] allow a prover to prove statements about values hidden in a commitment. Let Com be a commitment scheme. Given a commitment cm to $\mathbf{u}$, a CP-SNARK proves knowledge of a witness $\mathbf{w}$ consistent with cm and satisfying a relation $\mathcal{R}$. Formally, for $\mathcal{R}(\mathbf{x};\mathbf{w})$, we write $\mathbf{w} := (\omega,\mathbf{u},o)$, where $\mathbf{u}$ is committed, ω is uncommitted, and o is the opening randomness, such that $\mathsf{cm} = \mathsf{Com}(\mathbf{u};o)$ and $\mathcal{R}(\mathbf{x};\omega)=1$. A CP-SNARK is a tuple of algorithms $\Pi_{\mathsf{cp}} = (\mathsf{Setup},\mathsf{Prove},\mathsf{Verify})$.

- $(\mathsf{pk},\mathsf{vk}) \leftarrow \Pi_{\mathsf{cp}}.\mathsf{Setup}(1^\lambda,\mathcal{R},\mathsf{ck})$: Same as $\Pi.\mathsf{Setup}$, but additionally takes the commitment key ck as input.
- $\pi_{\mathsf{cp}} \leftarrow \Pi_{\mathsf{cp}}.\mathsf{Prove}(\mathsf{pk},\mathbf{x};\mathbf{w})$: Equal to $\Pi_{\mathsf{cp}}.\mathsf{Prove}(\mathsf{pk},\{\mathbf{x},\mathsf{cm}\};\{\mathbf{u},o\},\omega)$. It is based on $\Pi.\mathsf{Prove}$, but additionally takes the commitment cm as input, and the witness includes the committed value $\mathbf{u}$ and its opening o, together with the uncommitted part ω.
- $0/1 \leftarrow \Pi_{\mathsf{cp}}.\mathsf{Verify}(\mathsf{vk},\mathbf{x},\mathsf{cm},\pi_{\mathsf{cp}})$ Same as $\Pi.\mathsf{Verify}$, except that it additionally takes cm as input.

Commit-Carrying SNARKs (CC-SNARKs). A *Commit-Carrying SNARKs* (CC-SNARKs) [4, 14] extend the commit-and-prove approach by generating the witness commitment during proving. This helps scale to large circuits by moving expensive computations (e.g., group exponentiation) outside the circuit: the prover commits using a circuit-dependent key ck, and the resulting commitment cm is carried into the verification equation. A cc-SNARK consists of three PPT algorithms $\Pi_{\mathsf{cc}} = (\mathsf{Setup},\mathsf{Prove},\mathsf{Verify})$.

- $(\mathsf{ck},\mathsf{pk},\mathsf{vk}) \leftarrow \Pi_{\mathsf{cc}}.\mathsf{Setup}(1^\lambda,\mathcal{R})$: Similar to $\Pi_{\mathsf{cp}}.\mathsf{Setup}$, but additionally outputs the circuit-dependent commitment key ck together with $(\mathsf{pk},\mathsf{vk})$.
- $(\pi_{\mathsf{cc}},\tilde{\mathsf{cm}}) \leftarrow \Pi_{\mathsf{cc}}.\mathsf{Prove}(\mathsf{pk},\mathbf{x};\mathbf{w},\tilde{o})$: The witness is partitioned as $\mathbf{w} := (\mathbf{u},\omega)$ into a committed part and an uncommitted part. The prover outputs a proof π_{cc} together with a proof-dependent commitment $\tilde{\mathsf{cm}} = \mathsf{Com}.\mathsf{Commit}(\mathsf{ck},\mathbf{u};\tilde{o})$.
- $0/1 \leftarrow \Pi_{\mathsf{cc}}.\mathsf{Verify}(\mathsf{vk},\mathbf{x},\tilde{\mathsf{cm}},\pi_{\mathsf{cc}})$: On input the verification key vk, a statement $\mathbf{x}$, a proof-dependent commitment $\tilde{\mathsf{cm}}$, and a proof π_{cc}, the verifier outputs 0(reject) or 1(accept)

3.5 Freivalds' Algorithm

Freivalds' algorithm [11, 15] is a simple yet powerful randomized method for verifying a matrix product without recomputing it. Let $\mathbf{X},\mathbf{Y},\mathbf{Z} \in \mathbb{F}^{n\times n}$ and consider the claim $\mathbf{X}\cdot\mathbf{Y} = \mathbf{Z}$. Instead of computing $\mathbf{XY}$ in $O(n^3)$ time, Freivalds samples a uniformly random vector $\mathbf{r} \xleftarrow{\$} \mathbb{F}^n$ and checks

$$\mathbf{X}(\mathbf{Yr}) \stackrel{?}{=} \mathbf{Zr}.$$

This verification runs in $O(n^2)$ field operations. This algorithm has soundness error: if $\mathbf{XY} = \mathbf{Z}$, it always accepts. Otherwise, when $\mathbf{XY} \neq \mathbf{Z}$, the check passes with probability at most $1/|\mathbb{F}|$, i.e.,

$$\Pr\left[\,\mathbf{X}(\mathbf{Yr}) = \mathbf{Zr} \;\middle|\; \mathbf{XY} \neq \mathbf{Z}\,\right] \le \frac{1}{|\mathbb{F}|}.$$

4 Technical Overview

The main computational bottleneck in Transformers is the quadratic (or higher) cost of the self-attention matrix multiplications. To address this, recent work [1, 7, 25, 28, 29] proposes alternatives that reduce the cost of self-attention, with several methods achieving near-linear complexity while maintaining accuracy. One important line of research is motivated by the observation that the self-attention matrix is approximately low rank. By projecting the input sequence into a lower-dimensional subspace, these methods retain the essential information while reducing the computational cost to $O(n)$.

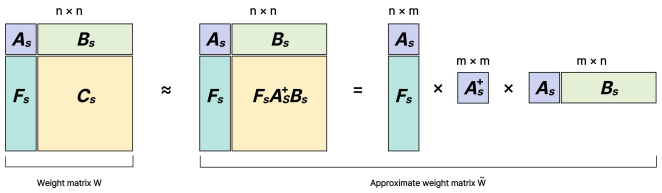
In the domain of zero-knowledge proofs, Freivalds' algorithm has been widely used to reduce the verification cost of matrix multiplication from $O(n^3)$ to $O(n^2)$. However, prior work has rarely explored how the inherent low-rank structure of self-attention can be exploited to further reduce the proof cost in zero-knowledge settings.

In this work, we propose **zkDietFormer**, which leverages the low-rank structure of self-attention to reduce proof cost to linear complexity. Our key idea is to avoid proving the full attention matrix and instead construct a zero-knowledge proof over its low-rank approximation. To address the $O(n^2)$ bottleneck and optimize proof efficiency, we examine several representative approaches:

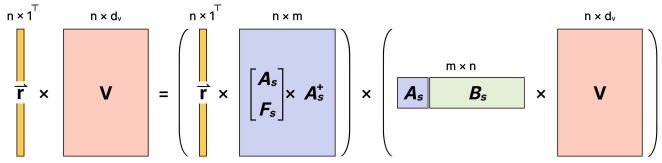
- Linformer is based on the observation that the self-attention matrix is low rank. It projects the keys and values into a lower-dimensional space using learned projection matrices, thereby achieving linear complexity.
- Nyströmformer applies the Nyström method to select representative landmark points from the full matrix and reconstruct the attention matrix from these landmarks.
- Performer uses the FAVOR+ algorithm to approximate attention via kernelization. Rather than projecting into a low-dimensional space, it decomposes attention as a product of low-dimensional matrices and rearranges computation to improve efficiency.

In our design, we adopt Nyströmformer, as it does not require introducing additional learned projection matrices into the proving

circuit, thereby simplifying the proof structure. Moreover, the theoretically grounded Nyström approximation provides numerically stable reconstruction regardless of sequence length.



(a) Nystromformer matrix approximation



(b) Our zkDiet Freivald Algorithm

Figure 1: Low rank matrix multiplication with Freivald Algorithm

As illustrated in Figure 1a, rather than directly handling the $n \times n$ self-attention matrix, we follow the Nyströmformer procedure and decompose it into a sequence of low-dimensional matrices of shapes $n \times m$, $m \times m$, and $m \times n$. By applying Freivalds' algorithm to this low-rank representation(in Figure 1b), we compress the verification workload from $O(n^2)$ to $O(n)$, effectively reducing it to vector multiplications of size $n + m^3 + nm^2 + mnd_v + m^3 + nmd_v$. This enables simultaneous reduction of both the prover's proof-generation time and the verifier's computational overhead.

To verify such low-rank matrix multiplications using Freivalds' algorithm, we adopt a Commit-and-Prove SNARK (CP-SNARK) architecture. (in Figure 2) The attention matrix is first committed outside the circuit, and the prover shows that the committed matrix is consistent with the private witness inside the circuit, while simultaneously proving that the matrix multiplication result is correct via a Freivalds algorithm with linear complexity. Instead of transmitting the full attention matrix, only the externally generated commitment is provided to the verifier, who then performs merely constant-time commitment checks. This design substantially reduces both proving and verification costs for matrix operations arising in large-scale models.

Because zero-knowledge circuits operate over finite fields, real-valued Transformer computations cannot be represented directly. We therefore quantize the computations in a manner that minimizes precision loss, leveraging the R1CS-friendly quantization technique proposed in [10].

5 zkDietFormer: Efficient Proofs of Low-Rank Matrix Multiplication via Freivalds' Algorithm

In this section, we present the core protocol of **zkDietFormer**. Our goal is to prove the correctness of Nyström -approximated low-rank self-attention while avoiding any explicit computation of the $n \times n$ attention matrix inside the circuit. Concretely, we design the protocol so that the number of constraints scales linearly with the sequence length n. To this end, we adopt a commit-and-prove SNARK architecture that pushes computationally expensive operations (e.g., large matrix multiplications and pseudoinverse computation) outside the circuit, and proves only their correctness inside the circuit using a small number of constraints.

5.1 Protocol Overview

Our protocol follows the message flow shown in Figure 3. First, the prover runs NyströmAttention on the input sequence $\mathbf{X} \in \mathbb{F}^{n \times d}$ and the model parameters $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{F}^{d \times d}$. This computation is performed outside the circuit, and thus can include heavy operations such as pseudoinverse computation. From this execution, the prover obtains the query, key, and value matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}$, where $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}^K$, and $\mathbf{V} = \mathbf{X}\mathbf{W}^V$. The prover then commits to $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ and sends the resulting commitments $\widetilde{\mathsf{cm}}_Q, \widetilde{\mathsf{cm}}_K, \widetilde{\mathsf{cm}}_V$ to the verifier.

Next, the verifier samples random challenge vectors $\mathbf{r}_n \in \mathbb{F}^n$ and $\mathbf{r}_k \in \mathbb{F}^k$ for Freivalds' randomized checks, and sends them to the prover. The prover sets the public instance as

$$\mathbf{x} := (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_k),$$

and generates a SNARK proof π using the proving key pk. The witness consists of a committed part

$$\mathbf{u} := (\mathbf{Q}, \mathbf{K}, \mathbf{V}),$$

and an uncommitted part

$$\omega := \left(\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V,\ \widetilde{\mathbf{Q}\mathbf{K}}^\top,\ \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top,\ \widetilde{\mathbf{Q}}\mathbf{K}^\top,\ \widetilde{A^+}\right).$$

Finally, the verifier checks the proof π against the public instance $\mathbf{x}$ and the previously received commitments $\widetilde{\mathsf{cm}}_Q, \widetilde{\mathsf{cm}}_K, \widetilde{\mathsf{cm}}_V$ using the verification key vk. The above interactive protocol can be made non-interactive via the Fiat–Shamir transformation.

5.2 Nyström -Based Low-Rank Self-Attention

Our system adopts a Nyström -based low-rank approximation for self-attention. In Nyströmformer , the query and key matrices are represented using k landmark vectors. Algorithm 1 describes the landmark selection procedure. Given $\mathbf{Q} = [\mathbf{q}_1, \ldots, \mathbf{q}_n]$ and $\mathbf{K} = [\mathbf{k}_1, \ldots, \mathbf{k}_n]$, We summarize the length-n sequence using k landmarks by partitioning it into k consecutive segments of length $\ell = n/k$ and defining each landmark as the average over the ℓ tokens in its segment. This yields $\widetilde{\mathbf{Q}} = [\tilde{\mathbf{q}}_1, \ldots, \tilde{\mathbf{q}}_k]$ and $\widetilde{\mathbf{K}} = [\tilde{\mathbf{k}}_1, \ldots, \tilde{\mathbf{k}}_k]$. With these landmarks, we avoid constructing the full $n \times n$ attention matrix and instead decompose the computation into three lower-dimensional matrices: $\mathbf{Q}\widetilde{\mathbf{K}}^\top \in \mathbb{F}^{n \times k}$, $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top \in \mathbb{F}^{k \times k}$, $\widetilde{\mathbf{Q}}\mathbf{K}^\top \in \mathbb{F}^{k \times n}$

We then apply the row-wise softmax to each of these matrices and compute

$$\tilde{F} := \mathrm{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d}}\right), \tilde{A} := \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d}}\right), \tilde{B} := \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d}}\right).$$

Here, $\tilde{A}$ is a small $k \times k$ matrix corresponding to the landmark submatrix of the original $n \times n$ attention matrix. Overall, the combination of $\tilde{F}$, $\tilde{A}$, and $\tilde{B}$ yields a Nyström approximation of the full attention map.

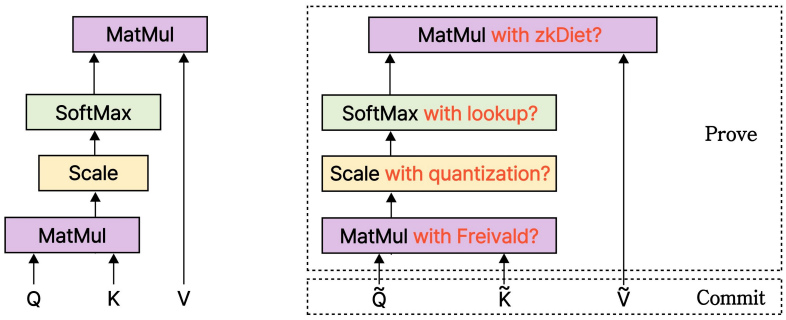


Figure 2: Overview of Self-Attention and our framework

Protocol

$\mathcal{P}(\mathsf{ck}, \mathbf{X}; \{W^Q, W^K, W^V\})$ $\qquad$ $\mathcal{V}(\mathsf{vk}, \mathbf{X})$

$\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{Q}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\mathbf{K}^\top, \widetilde{A}^+ \leftarrow \mathsf{NyströmAttention}(\mathbf{X}, W^Q, W^K, W^V)$

$(\widetilde{\mathsf{cm}}_Q, \widetilde{\mathsf{cm}}_K, \widetilde{\mathsf{cm}}_V) \leftarrow \mathsf{Commit}(\mathsf{ck}, \mathbf{Q}; o_Q^{\sim}),\ \mathsf{Commit}(\mathsf{ck}, \mathbf{K}; o_K^{\sim}),\ \mathsf{Commit}(\mathsf{ck}, \mathbf{V}; o_V^{\sim})$

$\widetilde{\mathsf{cm}}_Q, \widetilde{\mathsf{cm}}_K, \widetilde{\mathsf{cm}}_V \longrightarrow$

$\longleftarrow \mathbf{r}_n, \mathbf{r}_k$ $\qquad$ $\mathbf{r}_n \xleftarrow{\$} \mathbb{F}^n, \quad \mathbf{r}_k \xleftarrow{\$} \mathbb{F}^k$

$\mathbf{x} := \{\mathbf{X}, \mathbf{r}_n, \mathbf{r}_k\}$

$\mathbf{u} := \{\mathbf{Q}, \mathbf{K}, \mathbf{V}\}$

$\omega := \{W^Q, W^K, W^V, \mathbf{Q}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\mathbf{K}^\top, \widetilde{A}^+\}$

$\pi \leftarrow \Pi_{\mathsf{cc}}.\mathsf{Prove}(\mathsf{pk}, \mathbf{x}; (\mathbf{u}, \omega))$

$\pi \longrightarrow$

$\Pi_{\mathsf{cc}}.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}, \widetilde{\mathsf{cm}}_Q, \widetilde{\mathsf{cm}}_K, \widetilde{\mathsf{cm}}_V, \pi)$

Figure 3: Our protocol

Finally, we compute an approximation to the Moore–Penrose pseudoinverse $\tilde{A}^+$ of the $k \times k$ matrix $\tilde{A}$. While one could compute $\tilde{A}^+$ via SVD, this is typically inefficient on GPU hardware. Therefore, Nyströmformer approximates $\tilde{A}^+$ via an iterative polynomial approximation to the SVD. Algorithm 2 shows this iterative procedure.

Using the resulting $\tilde{A}^+$, we form the final approximated attention matrix

$$\hat{S} := \tilde{F} \cdot \tilde{A}^+ \cdot \tilde{B},$$

and compute the attention output as

$$\mathbf{Y} := \hat{S} \cdot \mathbf{V}.$$

Algorithm 3 provides the complete Nyström -based self-attention computation used in our framework.

5.3 Efficient Verification via Freivalds' Algorithm

Our protocol uses Freivalds' algorithm as a core tool to efficiently verify that the low-rank self-attention computation is performed correctly. In particular, within the zk-SNARK circuit, we employ Freivalds' algorithm to ensure that the following key operations

Algorithm 1 Select landmarks

Input: Query $\mathbf{Q}$ and Key $\mathbf{K} \in \mathbb{F}^{n \times d}$
Output: Landmark points $\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \in \mathbb{F}^{k \times d}$

SelectLandmarks$(\mathbf{Q}, \mathbf{K})$:

1: **parse** $\mathbf{Q} = [\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n], \mathbf{K} = [\mathbf{k}_1, \mathbf{k}_2, \ldots, \mathbf{k}_n]$
2: $\ell \leftarrow n/k$
3: **for** $j \in \{1, \ldots, k\}$ **do**
4: $\quad \tilde{\mathbf{q}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{q}_i}{\ell}, \tilde{\mathbf{k}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{k}_i}{\ell}$
5: **endfor**
6: $\widetilde{\mathbf{Q}} \leftarrow [\tilde{\mathbf{q}}_1, \tilde{\mathbf{q}}_2, \ldots, \tilde{\mathbf{q}}_k]$
7: $\widetilde{\mathbf{K}} \leftarrow [\tilde{\mathbf{k}}_1, \tilde{\mathbf{k}}_2, \ldots, \tilde{\mathbf{k}}_k]$
8: **return** $(\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}})$

Algorithm 2 Compute Moore–Penrose inverse

Input: selected landmark matrix $\tilde{A} \in \mathbb{F}^{k\times k}$
Output: approximate pseudoinverse $\tilde{A}^{+} \in \mathbb{F}^{k\times k}$

ComputeMPInverse($\tilde{A}$):

$Z_0 \leftarrow \tilde{A}^{\top} / \left(\|\tilde{A}\|_1 \cdot \|\tilde{A}\|_\infty \right)$
Set T (e.g., $T = 6$ iterations)
for $t \in \{0, \ldots, T-1\}$ **do**
  $Z_{t+1} \leftarrow \frac{1}{4} Z_t \left(13I - \tilde{A} Z_t \left(15I - \tilde{A} Z_t \left(7I - \tilde{A} Z_t \right) \right) \right)$
endfor
$\tilde{A}^{+} \leftarrow Z_T$
return $\tilde{A}^{+}$

Algorithm 3 Nyström approximation of attention matrix

Input: Input sequence $\mathbf{X} \in \mathbb{F}^{n\times d}$,
  Model parameters $W^{Q}, W^{K}, W^{V} \in \mathbb{F}^{d\times d}$
Output: Attention matrix $\mathbf{Y} := \hat{\mathbf{S}}\mathbf{V} \in \mathbb{F}^{n\times d}$,
  Auxiliary $aux := \mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{Q}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\mathbf{K}^{\top}, \widetilde{A}^{+}$

NyströmAttention($\mathbf{X}, W^{Q}, W^{K}, W^{V}$):

$\mathbf{Q} := \mathbf{X} \cdot W^{Q}$
$\mathbf{K} := \mathbf{X} \cdot W^{K}$
$\mathbf{V} := \mathbf{X} \cdot W^{V}$
$\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \leftarrow$ SelectLandmarks($\mathbf{Q}, \mathbf{K}$)
$\tilde{F} := \sigma\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^{\top}}{\sqrt{d}} \right), \tilde{A} := \sigma\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}}{\sqrt{d}} \right), \tilde{B} := \sigma\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^{\top}}{\sqrt{d}} \right)$
$\tilde{A}^{+} \leftarrow$ ComputeMPInverse($\tilde{A}$)
$\hat{S} := \tilde{F} \cdot \tilde{A}^{+} \cdot \tilde{B}$
$\mathbf{Y} := \hat{S} \cdot \mathbf{V} = \tilde{F} \cdot \tilde{A}^{+} \cdot \tilde{B} \cdot \mathbf{V}$
$aux := \mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{Q}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\mathbf{K}^{\top}, \widetilde{A}^{+}$
return $\mathbf{Y}, aux$

can be verified with linear cost.

Query/Key/Value matrix generation. Using the verifier's random vector $\mathbf{r}_n$, we check the relation $\mathbf{Q} = \mathbf{X}\mathbf{W}^{Q}$ by comparing $\mathbf{r}_n^{\top}\mathbf{Q}$ with $\mathbf{r}_n^{\top}(\mathbf{X}\mathbf{W}^{Q})$. We verify $\mathbf{K}$ and $\mathbf{V}$ in the same way. Rather than proving the full $n \times d$ matrix product directly, this Freivalds' check reduces verification to computing only the vector-matrix product $\mathbf{r}_n^{\top}\mathbf{X}W^{Q}$, thereby reducing the complexity to $O(n)$.

Low-rank matrix operations. We also verify low-dimensional products such as $\mathbf{Q}\widetilde{\mathbf{K}}^{\top}$, $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}$, and $\widetilde{\mathbf{Q}}\mathbf{K}^{\top}$ by multiplying each matrix with a random vector of matching dimension, i.e., $\mathbf{r}_n$ or $\mathbf{r}_k$. For instance, we can certify the correctness of $\mathbf{Q}\widetilde{\mathbf{K}}^{\top}$ by checking the following equality:

$$\mathbf{r}_n^{\top}(\mathbf{Q}\widetilde{\mathbf{K}}^{\top}) \stackrel{?}{=} (\mathbf{r}_n^{\top}\mathbf{Q})\,\widetilde{\mathbf{K}}^{\top}$$

Similarly, the products $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}$ and $\widetilde{\mathbf{Q}}\mathbf{K}^{\top}$ are validated with linear cost, without explicitly computing the full matrix multiplications inside the circuit.

Relation

1: $\mathcal{R}(\mathbf{x}, \mathbf{w})$:
2: **parse** $\mathbf{x} = (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_k), \mathbf{w} = (\mathbf{u}, \omega)$
3: **parse** $\mathbf{u} = (\mathbf{Q}, \mathbf{K}, \mathbf{V})$
4: **parse** $\omega = (W^{Q}, W^{K}, W^{V}, \mathbf{Q}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\mathbf{K}^{\top}, \widetilde{A}^{+})$
5: **parse** $\mathbf{Q} = [\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n], \mathbf{K} = [\mathbf{k}_1, \mathbf{k}_2, \ldots, \mathbf{k}_n]$
6: $\mathbf{r}_n \cdot \mathbf{Q} = \mathbf{r}_n \cdot \mathbf{X} \cdot W^{Q}$
7: $\mathbf{r}_n \cdot \mathbf{K} = \mathbf{r}_n \cdot \mathbf{X} \cdot W^{K}$
8: $\mathbf{r}_n \cdot \mathbf{V} = \mathbf{r}_n \cdot \mathbf{X} \cdot W^{V}$
9: **for** $j \in \{1, \ldots, k\}$ **do**
10: $\tilde{\mathbf{q}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{q}_i}{\ell}, \tilde{\mathbf{k}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{k}_i}{\ell}$
11: **endfor**
12: $\widetilde{\mathbf{Q}} \leftarrow [\tilde{\mathbf{q}}_1, \ldots, \tilde{\mathbf{q}}_k], \widetilde{\mathbf{K}} \leftarrow [\tilde{\mathbf{k}}_1, \ldots, \tilde{\mathbf{k}}_k]$
13: $\mathbf{r}_n \cdot \mathbf{Q}\widetilde{\mathbf{K}}^{\top} = \mathbf{r}_n \cdot \mathbf{Q} \cdot \widetilde{\mathbf{K}}^{\top}$
14: $\mathbf{r}_k \cdot \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top} = \mathbf{r}_k \cdot \widetilde{\mathbf{Q}} \cdot \widetilde{\mathbf{K}}^{\top}$
15: $\mathbf{r}_n \cdot \widetilde{\mathbf{Q}}\mathbf{K}^{\top} = \mathbf{r}_n \cdot \widetilde{\mathbf{Q}} \cdot \mathbf{K}^{\top}$
16: $\widetilde{F} = \sigma\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^{\top}}{\sqrt{d}} \right)$
17: $\widetilde{A} = \sigma\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}}{\sqrt{d}} \right)$
18: $\widetilde{B} = \sigma\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^{\top}}{\sqrt{d}} \right)$
19: $\mathbf{r}_k \cdot \widetilde{A} \cdot \widetilde{A}^{+} \cdot \widetilde{A} = \mathbf{r}_k \cdot \widetilde{A}$
20: $\mathbf{r}_k \cdot \widetilde{A}^{+} \cdot \widetilde{A} \cdot \widetilde{A}^{+} = \mathbf{r}_k \cdot \widetilde{A}^{+}$
21: $\mathbf{r}_k \cdot (\widetilde{A}\widetilde{A}^{+})^{\top} = \mathbf{r}_k \cdot \widetilde{A}\widetilde{A}^{+}$
22: $\mathbf{r}_k \cdot (\widetilde{A}^{+}\widetilde{A})^{\top} = \mathbf{r}_k \cdot \widetilde{A}^{+}\widetilde{A}$
23: $\mathbf{r}_n \cdot \mathbf{Y} = \mathbf{r}_n \cdot \widetilde{F} \cdot \widetilde{A}^{+} \cdot \widetilde{B} \cdot \mathbf{V}$

Figure 4: Relation of zkDietFormer

Moore–Penrose pseudoinverse. Computing the Moore–Penrose pseudoinverse $\tilde{A}^{+}$ inside the circuit is inefficient. Instead, we certify that $\tilde{A}^{+}$ is the pseudoinverse of $\tilde{A}$ by checking the four Moore–Penrose conditions, which correspond to lines 19–22 in Figure 4:

- $\tilde{A} \cdot \tilde{A}^{+} \cdot \tilde{A} = \tilde{A}$
- $\tilde{A}^{+} \cdot \tilde{A} \cdot \tilde{A}^{+} = \tilde{A}^{+}$
- $(\tilde{A}\tilde{A}^{+})^{\top} = \tilde{A}\tilde{A}^{+}$
- $(\tilde{A}^{+}\tilde{A})^{\top} = \tilde{A}^{+}\tilde{A}$

This allows the prover to efficiently demonstrate that $\tilde{A}^{+}$ is a valid Moore–Penrose pseudoinverse of $\tilde{A}$ without explicitly computing it in-circuit.

The relations that the prover must satisfy are formalized in Figure 4 as the definition of the circuit relation. In particular, the relation $\mathcal{R}(\mathbf{x}, \mathbf{w})$ takes the public instance $\mathbf{x} = (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_k)$ and the witness $\mathbf{w} = (\mathbf{Q}, \mathbf{K}, \mathbf{V}, W^{Q}, W^{K}, W^{V}, \mathbf{Q}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\mathbf{K}^{\top}, \widetilde{A}^{+})$, and enforces all required equalities as well as the softmax computation. All key equalities are expressed in the form of products with the random vectors, accordingly, the circuit computes only vector inner products.

6 Security

In this section, we show that zkDietFormer satisfies the completeness and security requirements. Since zkDietFormer is built upon the cc-Groth16 protocol, it inherits its guarantees of completeness, soundness, and zero-knowledge. **Completeness**. If an honest prover correctly performs the Nyströmformer-based linear attention computation and generates a proof, then the verifier always accepts it. In our system, the only potential source of completeness error arises from nonlinear operations handled via lookup tables (i.e., exponentiation operations). For matrix operations, the Freivalds' algorithm always returns true when the equation is correct. In the case of lookup tables, one may theoretically consider situations where the denominator becomes zero. However, **zkDiet** avoids this issue through quantization with zero-point adjustment, which maps all inputs into the positive domain. As a result, any scenario in which the denominator is approximated as zero is mathematically eliminated, preserving completeness.

Soundness. Even a malicious prover cannot convince the verifier of an incorrect matrix computation except with negligible probability. Our system satisfies two essential security properties.

First, the integrity of the matrix multiplication $AB = C$ is guaranteed by Freivalds' algorithm. When $AB \neq C$, the probability that a randomly chosen vector r satisfies $ABx = C$ is extremely small, bounded by $O(1/|\mathbb{F}|)$, which is inversely proportional to the size of the finite field $\mathbb{F}$. To simplify the analysis, we assume in the following proof that the random vector r used in Freivalds' algorithm is sampled from the binary domain $\{0,1\}^n$.

Proof. Since $\hat{D}^{-1}$ is a diagonal matrix, the overall complexity $O(nkd)$ is determined by the $\tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V})$ computation, excluding the cost of forming $\hat{D}^{-1}$. In the following section, we demonstrate that the Freivalds' algorithm correctly verifies the computation of $\tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V})$. Suppose the prover outputs a matrix y' that has a different **structure or value** from the correct output $y = \tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V})$. Let

$$s_1' = \vec{r}\, y', \qquad s_2 = (\vec{r}\, \tilde{\mathbf{F}}\mathbf{A}'^{+})((\mathbf{B}')^{\top}\mathbf{V}).$$

where $\tilde{\mathbf{F}}\mathbf{A}'^{+}, \mathbf{B}' \in \mathbb{R}^{n\times k}, \mathbf{V} \in \mathbb{R}^{n\times d}, \vec{r} \in \{0,1\}^n$. Let ϵ denote the error probability that the Freivalds verification outputs TRUE even when it is incorrect. We claim that if $\tilde{\mathbf{F}}\mathbf{A}'^{+} \times ((\mathbf{B}')^{\top})\mathbf{V} = y'$, then $\epsilon = 0$, and if $\tilde{\mathbf{F}}\mathbf{A}'^{+} \times ((\mathbf{B}')^{\top})\mathbf{V} \neq y'$, then $\epsilon \leq \frac{1}{2}$.

- **Case** $\tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V}) = y'$ **:** The error matrix $D = \tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V}) - y'$ is $0_{n\times d}$. Regardless of the choice of the vector $\vec{r}$, the equality is always satisfied because $\vec{r}D = \vec{0}$. Hence,

$$\epsilon = \Pr[\vec{r}D \neq 0] = \Pr[\vec{r}0 \neq \vec{0}] = 0$$

  Therefore, the probability of error is $\epsilon = 0$.

- **Case** $\tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V}) \neq y'$ **:** Let $\vec{P} = \vec{r}D = (p_1, p_2, ..., p_d)$, where $D = \tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V}) - y'$. Since $\tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V}) \neq y'$, there exists at least one nonzero element in D. Let $d_{i^*,j}$ be the first nonzero entry in column j. For the value $\Delta = \sum_{k\neq i^*} r_k d_{kj}$. Using Bayes' Theorem, we can partition over Δ.

$$p_j = \sum_{k=1}^{n} r_k d_{kj} = r_{i^*} d_{i^*j} + \sum_{k\neq i^*} r_k d_{kj} = r_{i^*} d_{i^*j} + \Delta. \tag{1}$$

$$\begin{aligned}\Pr[p_j = 0] = \Pr[p_j = 0|\Delta = 0] \cdot \Pr[\Delta = 0] \\ + \Pr[p_j = 0|\Delta \neq 0] \cdot \Pr[\Delta \neq 0].\end{aligned} \tag{2}$$

$$\Pr[p_j = 0|\Delta = 0] = \Pr[r_{i^*} = 0] = \frac{1}{2}. \tag{3}$$

For $\Pr[p_j = 0|\Delta \neq 0]$, if $r_{i^*} d_{i^*j} + \Delta = 0$, it holds as long as $r_{i^*} = 1$ (among the possible values of $r_{i^*} \in \{0,1\}$)so

$$\Pr[p_j = 0|\Delta \neq 0] = \Pr[r_{i^*} = 1] = \frac{1}{2}$$

if $d_{i^*j} + \Delta \neq 0$, then r_{i^*} cannot take any value to satisfy the condition, so $\Pr[p_j = 0|\Delta \neq 0] = 0$. Therefore, $\Pr[p_j = 0|\Delta \neq 0] \leq \frac{1}{2}$.
After taking this case into account, the probability can be rewritten as follows:

$$\begin{aligned}\Pr[p_j = 0] &\leq \frac{1}{2} \cdot \Pr[\Delta = 0] + \frac{1}{2} \cdot \Pr[\Delta \neq 0] \\ &= \frac{1}{2} \cdot \Pr[\Delta = 0] + \frac{1}{2} \cdot (1 - \Pr[\Delta = 0]) \\ &= \frac{1}{2}.\end{aligned} \tag{4}$$

$$\epsilon = \Pr[\vec{P} = 0 \mid y' \neq y]. \tag{5}$$

$$\begin{aligned}\epsilon &= \Pr[\vec{P} = 0] \\ &= \Pr[p_1 = 0 \wedge \cdots \wedge p_j = 0 \wedge \cdots \wedge p_d = 0] \\ &\leq \Pr[p_j = 0] \leq \frac{1}{2}.\end{aligned} \tag{6}$$

$$\epsilon \leq \frac{1}{2}. \tag{7}$$

After t independent repetitions, the ϵ becomes $(\frac{1}{2})^t$, which is negligible in t. Therefore, the verification protocol is sound: the probability that it outputs TRUE when $\tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V}) \neq y'$ is $\epsilon \leq (\frac{1}{2})^t$. □

Second, the strong binding property of the KZG polynomial commitment ensures that the prover cannot claim values different from those already committed. Using the commit-and-prove capability of CP-SNARK, we prove (1) correctness of matrix multiplication and (2) consistency of the Softmax outputs, while cp-link additionally guarantees (3) input consistency across these two computations. Consequently, any attempt to replace a committed value with another one cannot pass verification.

Zero-Knowledge. **zkDiet** reveals no secret information during inference, including model weights, inputs, or intermediate activations. All values are handled in committed form. The verifier learns only that the linear transformer computation was carried out correctly, while gaining no additional information about the committed values.

7 Experiments

In this section, we evaluate the concrete proof generation costs of **zkDietFormer** and compare them against a standard self-attention baseline. Because our contribution targets the architectural cost of proving attention—i.e., the circuit structure rather than the learned weights—the evaluation focuses on the cost of the circuit itself. We use modified HuggingFace BERT and Nystromformer configurations (modified hidden dimensions and layer counts) to enable proving across multiple model sizes; consequently, model inference accuracy is not a focus of this evaluation.

7.1 Implementation

We implement **zkDietFormer** in two parts: a Python-based witness generation pipeline and a Rust-based zero-knowledge proving circuit. Witness data (model weights and intermediate activations) are generated using PyTorch 3.11.15 and HuggingFace Transformers 5.8.0, which instrument the standard `BertForMaskedLM` and `NystromformerForMaskedLM` model classes with forward hooks to capture per-layer activations. The Rust circuit is built on the arkworks library over the BLS12-381 elliptic curve and uses the BLAKE2s hash function for cryptographic operations.

We use LegoGroth16 [4] as our commit-and-prove SNARK (CP-SNARK) instantiation. Non-arithmetic operations—softmax exponentials, SigmoidGELU activation, and LayerNorm inverse square root—are evaluated using Lasso [21] lookup arguments.

Pseudoinverse Verification. The Nyströmformer attention mechanism requires computing the Moore–Penrose pseudoinverse $K^{\dagger}$ of a small $m \times m$ matrix K derived from the softmax of landmark interactions, where m is the number of landmarks. The reference implementation [29] computes this pseudoinverse via the iterative Newton–Schulz method: starting from an initial estimate $V_0 = \|K\|_1^{-1} K^{\top}$, the iteration

$$V_{i+1} = \tfrac{1}{4} V_i \big(13\, I - K V_i \big(15\, I - K V_i \big(7\, I - K V_i\big)\big)\big)$$

is applied for, for example, $n_{\text{iter}} = 6$ rounds. Directly arithmetizing this cubic Newton iteration is highly expensive: each round requires three sequential matrix multiplications, yielding 18 matrix multiplications in total per layer, all of which would need to be encoded as R1CS constraints.

Instead of proving the entire iterative computation in-circuit, we adopt a witness-supplied verification approach: the prover computes $K^{\dagger}$ outside the circuit using the Newton–Schulz iteration as in the reference implementation, then commits the result V as a witness. The circuit then verifies that V is indeed the pseudoinverse of K by checking the four defining Moore–Penrose conditions, using the Freivalds-based verification introduced in Section 5.3. Since $m = 32$ is small and independent of the sequence length n, this verification adds only a constant overhead regardless of input length.

Quantization. Zero-knowledge circuits operate over finite fields and cannot represent floating-point arithmetic directly. We therefore quantize all model computations following the R1CS-friendly quantization scheme of ZEN [10]. All weights and activations are converted to 8-bit asymmetric fixed-point representations, and the circuit precision parameter is set to $k_{\text{bits}} = 22$, providing sufficient headroom for accumulated multiplication errors across a full layer. Different operators use different fractional precisions to maintain numerical fidelity: LayerNorm uses 8-bit fractional precision (`FRAC_BITS` = 8); Softmax uses 16-bit fractional precision (`FRAC_BITS` = 16), reflecting the higher dynamic range required by the exponential function; and GELU/Sigmoid are evaluated via lookup tables of size $2^{16} = 65{,}536$ entries, embedded into the circuit using lookup arguments.

Activation Function Replacement. While our primary focus is on the attention sub-circuit, we implemented and measured the full transformer pipeline (Attention + MLP + LayerNorm); detailed MLP + LayerNrom measurements are reported in Appendix A. One modification required by this end-to-end implementation concerns the GELU activation used in the MLP block. Both BERT and Nyströmformer use GELU [13] as their default activation, typically implemented via the approximation $\text{GELU}(x) \approx 0.5\, x\big(1 + \tanh\big[\sqrt{2/\pi}\,(x + 0.044715\, x^3)\big]\big)$. We replace GELU in both models with SigmoidGELU, defined as

$$\text{SigmoidGELU}(x) = x \cdot \sigma(1.702\, x),$$

where $\sigma(\cdot)$ denotes the logistic sigmoid. This alternative, originally noted by Hendrycks and Gimpel [13] as a close approximation to GELU, is substantially more circuit-friendly because the sigmoid function can be evaluated via a single lookup table, whereas the tanh-based form requires a more involved computational procedure. The replacement is applied to all layers in both models.

7.2 Experimental Configuration

Hardware. All experiments are conducted on a Mac Studio equipped with an Apple M1 Ultra chip (20 cores: 16 performance + 4 efficiency) and 128 GB of unified memory, running macOS 14.6.1. No GPU acceleration is used, as the underlying ZK library operates exclusively on CPU. All reported timing measurements are CPU wall-clock time using Rayon's default thread pool, which utilizes all available CPU cores (20 cores on our platform). Peak memory is measured as the maximum resident set size (RSS) via the system `time` utility.

Baselines. We adopt two publicly available Transformer architectures from the HuggingFace Transformers library [27]:

- **BERT** (`bert-base-uncased`, Devlin et al. [9]): employs standard scaled dot-product self-attention with $O(n^2 d)$ complexity.
- **Nyströmformer** (`uw-madison/nystromformer-512`, Xiong et al. [29]): replaces the full attention matrix with a Nyström low-rank approximation, reducing attention complexity to $O(n)$.

Both models are instantiated using the standard HuggingFace classes (`BertForMaskedLM` and `NystromformerForMaskedLM`) without any custom architectural modifications beyond the activation replacement and quantization.

Model Configurations. We evaluate five configurations spanning the standard BERT model family, from the smallest (Tiny) to the full-scale (Base) model. The configurations are summarized in Table 1. Here, L denotes the number of Transformer layers, d the hidden dimension, and A the number of attention heads. For each model scale, the BERT baseline and Nyströmformer variant share identical structural parameters (L, d, A), differing only in the attention kernel.

Model	L	d	A
Tiny	2	128	2
Mini	4	256	4
Small	4	512	8
Medium	8	512	8
Base	12	768	12

Table 1: Model configurations across the standard BERT family (Tiny to Base)

7.3 Results

All reported metrics are *per single layer*: each table entry is the average of the per-layer measurements across all L layers of the model (e.g., L=2 for Tiny). Constraint counts are identical across layers and reported directly. Timing values (setup, commit, prove, verify) and proof size are averaged over the L layers. Memory denotes the peak resident set size (RSS) observed during a single process that proves all L layers sequentially; because it is a process-wide maximum, it is *not* averaged but reported as a single value. We measure six sequence lengths $n \in \{128, 256, 512, 1024, 2048, 4096\}$ with a fixed landmark count m=32 for Nyström attention. This range is chosen to expose the asymptotic advantage of Nyström attention over standard self-attention: since the landmark count m is held fixed while n varies, the Nyström approximation scales as $O(n)$ in both time and memory [29], whereas standard self-attention scales as $O(n^2)$. The cost gap between the two thus widens as n grows, and our sweep is designed to capture this transition—from short sequences where the two are comparable, to long sequences where the quadratic baseline becomes prohibitively expensive. For each circuit instance, we report constraint count, setup time, prove time, verify time, peak memory, and two commitment costs: the Groth16 committing time (G16) for committing to the circuit witness, and the Lasso committing time for committing to the lookup operands. The additional Lasso commitment used for CP-Link bridging (*commit_cplink_link_ms*) is measured separately and discussed in the per-table analysis below. Runs that exceed available memory are reported as out-of-memory (OOM); these patterns are themselves informative results, as they indicate the practical scalability boundary of each circuit variant. We evaluate **zkDietFormer**by applying its two constituent techniques—low-rank (Nyström) attention and Freivalds-based matrix-multiplication verification—one at a time, first isolating each technique's individual contribution and then combining both to measure the advantage of the full scheme.

7.3.1 Low-Rank Attention is Essential for Asymptotic Scaling. To isolate the contribution of the attention kernel, we fix the verification method and compare the two kernels. Table 2 holds verification at naive (in-circuit matrix multiplication) and compares BERT vs. Nyström attention on the Tiny configuration (L=2, d=128, A=2).

Under naive verification, the $O(n^2)$-versus-$O(n)$ gap is modest at short sequences but widens rapidly. At n=128 BERT-naive requires 4.78M per-layer constraints versus 3.12M for Nyström-naive (1.5×). By n=256 the ratio widens to 19.07M vs. 5.78M (3.3×), with per-layer setup time diverging from 393.03 s to 106.56 s (3.7×) and prove time from 73.63 s to 22.14 s (3.3×). BERT-naive is out of memory at n≥512, whereas Nyström-naive survives until n=2048 (43.05M constraints, 867.26 s per-layer setup) before exhausting memory at n=4096. The only metric on which Nyström -naive is slower than BERT-naive is verify time (622.0 ms vs. 273.5 ms) at n=128. This residual gap reflects a constant overhead in our Nyström pipeline: the Nyström circuit must additionally verify four Moore–Penrose conditions and three separate row-wise softmax operations, each of which adds a fixed per-layer cost that is independent of n. As n grows, BERT's $O(n^2)$ contribution overtakes these constants and the advantage shifts decisively to Nyström across every metric. Since both configurations use identical naive verification and differ only in their attention kernel, this widening gap confirms that the $O(n)$ versus $O(n^2)$ asymptotics of the two attention mechanisms translate directly into the asymptotics of their ZK proof generation cost. Furthermore, the dedicated Lasso commitment time required for CP-Link interoperability (excluded from the table) remains modest at the sequence lengths where both configurations complete: 109.0 ms (BERT) vs. 145.5 ms (Nyström) at n=128, and 237.5 ms vs. 212.0 ms at n=256. Since BERT-naive exhausts memory at n≥512, only Nyström -naive CP-Link costs are observable beyond that point, growing from 283.0 ms (n=512) to 603.5 ms (n=2048).

7.3.2 Freivalds Removes the Constant Factor. We now hold the attention kernel fixed at Nyström and compare verification methods. Table 3 compares two ways of proving the matrix multiplications inside Nyström attention: a *naive* implementation that re-derives each matrix product inside the circuit, the most direct way of arithmetizing matrix multiplication, and our *Freivalds-based* verification, in which the prover commits the result outside the circuit and the circuit checks a randomized identity. For a product of two s×s matrices, this randomized check costs only $O(s^2)$ field operations—one order lower than the $O(s^3)$ cost of materializing the product inside the circuit. The two columns share the same low-rank attention structure; they differ only in how matrix multiplications are verified, isolating Freivalds' contribution.

Across all measured sequence lengths, Freivalds verification reduces the Nyström per-layer constraint count by a consistent factor of 5.8–6.3×. At n=256, for example, the per-layer constraint count drops from 5.78M (Naive) to 0.96M (Freivalds); per-layer setup time from 106.56 s to 32.42 s (3.3×); and prove time from 22.14 s to 7.78 s (2.8×). The reduction ratio remains stable across all sequence lengths, confirming that Freivalds provides a uniform constant-factor improvement independent of n. Without Freivalds, Nyström-naive reaches 43.05M per-layer constraints at n=2048 and exhausts our 128 GB memory at n=4096; with Freivalds the circuit completes all six sequence lengths. The CP-Link commitment cost (excluded from the table) is nearly identical between the two verification strategies: 145.5 ms (Naive) vs. 150.5 ms (Freivalds) at n=128, and 603.5 ms vs. 597.0 ms at n=2048. This is expected, as the CP-Link bridging phase depends on the number of committed elements rather than the internal circuit structure, confirming that Freivalds introduces no additional overhead in the commitment pipeline.

7.3.3 Freivalds Alone Cannot Substitute for Low-Rank Attention. Freivalds is equally effective on standard BERT attention: comparing the BERT columns of Tables 2 and 4, the per-layer constraint count drops from 4.78M to 0.72M at n=128 and from 19.07M

Table 2: Kernel comparison under *naive* matrix-multiplication verification (Tiny, L=2). All metrics are per single layer (averaged over L layers). OOM entries (—) indicate memory exhaustion at 128 GB. Proof Size excludes CP-Link public instances.

	n=128		n=256		n=512		n=1024		n=2048		n=4096	
	BERT	Nys.	BERT	Nys.	BERT	Nys.	BERT	Nys.	BERT	Nys.	BERT	Nys.
Constr. (M)	4.78	3.12	19.07	5.78	—	11.10	—	21.75	—	43.05	—	—
Setup (s)	95.21	57.34	393.03	106.56	—	208.16	—	415.85	—	867.26	—	—
G16 Cmt (ms)	46.5	50.5	147.0	73.0	—	140.5	—	261.0	—	826.0	—	—
Lasso Cmt (ms)	98.0	202.0	203.0	235.0	—	292.5	—	405.5	—	628.0	—	—
Prove (s)	17.06	13.75	73.63	22.14	—	41.73	—	86.06	—	294.71	—	—
Verify (ms)	273.5	622.0	417.5	770.5	—	792.5	—	1,088.0	—	1,176.0	—	—
Proof Size (KB)	374.22	1,018.28	499.38	1,090.81	—	1,180.34	—	1,307.12	—	1,466.91	—	—
Peak RAM (GB)	21.28	15.24	79.63	25.44	113.16	46.29	113.04	91.21	113.09	112.52	113.18	114.12

Table 3: Nyström attention: naive vs. Freivalds (Tiny, L=2). All metrics are per single layer (averaged over L layers). OOM entries (—) indicate memory exhaustion at 128 GB. Proof Size excludes CP-Link public instances.

	n=128		n=256		n=512		n=1024		n=2048		n=4096	
	Naive	Freiv.	Naive	Freiv.	Naive	Freiv.	Naive	Freiv.	Naive	Freiv.	Naive	Freiv.
Constr. (M)	3.12	0.54	5.78	0.96	11.10	1.79	21.75	3.47	43.05	6.83	—	13.54
Setup (s)	57.34	17.09	106.56	32.42	208.16	54.03	415.85	108.52	867.26	222.05	—	425.36
G16 Cmt (ms)	50.5	45.0	73.0	74.5	140.5	137.0	261.0	252.5	826.0	464.0	—	872.5
Lasso Cmt (ms)	202.0	189.0	235.0	233.5	292.5	280.0	405.5	405.0	628.0	614.0	—	945.5
Prove (s)	13.75	5.60	22.14	7.78	41.73	10.62	86.06	17.54	294.71	32.01	—	63.74
Verify (ms)	622.0	616.5	770.5	759.0	792.5	789.0	1,088.0	1,073.0	1,176.0	1,141.5	—	1,717.0
Proof Size (KB)	1,018.28	1,018.28	1,090.81	1,090.81	1,180.34	1,180.34	1,307.12	1,307.12	1,466.91	1,466.91	—	1,699.94
Peak RAM (GB)	15.24	3.70	25.44	7.24	46.29	11.12	91.21	18.81	112.52	34.63	114.12	69.02

Table 4: Kernel comparison under *Freivalds* matrix-multiplication verification (Tiny, L=2). All metrics are per single layer (averaged over L layers). OOM entries (—) indicate memory exhaustion at 128 GB. Proof Size excludes CP-Link public instances.

	n=128		n=256		n=512		n=1024		n=2048		n=4096	
	BERT	Nys.	BERT	Nys.	BERT	Nys.	BERT	Nys.	BERT	Nys.	BERT	Nys.
Constr. (M)	0.72	0.54	2.69	0.96	10.36	1.79	40.64	3.47	—	6.83	—	13.54
Setup (s)	24.28	17.09	95.42	32.42	357.85	54.03	1,476.13	108.52	—	222.05	—	425.36
G16 Cmt (ms)	46.5	45.0	166.0	74.5	462.5	137.0	3,099.5	252.5	—	464.0	—	872.5
Lasso Cmt (ms)	93.5	189.0	226.5	233.5	677.5	280.0	2,303.5	405.0	—	614.0	—	945.5
Prove (s)	3.86	5.60	12.60	7.78	46.68	10.62	312.63	17.54	—	32.01	—	63.74
Verify (ms)	275.5	616.5	412.0	759.0	710.5	789.0	1,350.0	1,073.0	—	1,141.5	—	1,717.0
Proof Size (KB)	374.22	1,018.28	499.38	1,090.81	730.56	1,180.34	1,163.66	1,307.12	—	1,466.91	—	1,699.94
Peak RAM (GB)	4.57	3.70	16.29	7.24	54.68	11.12	113.21	18.81	113.50	34.63	113.21	69.02

to 2.69M at n=256—a 6.6–7.1× reduction, comparable to the 5.8–6.3× observed for Nyström in Table 3. The slightly higher ratio for BERT reflects the larger fraction of total constraints contributed by matrix multiplication in standard self-attention. This confirms that Freivalds is a generic optimization rather than an artifact of Nyström structure. Yet even with Freivalds applied, BERT attention still exhausts our 128 GB memory at n≥2048 (Table 4), because the underlying $O(n^2)$ complexity is left intact. These results suggest that the constant-factor reduction from Freivalds and the asymptotic reduction from low-rank attention provide complementary benefits, making their combination effective for improving scalability across a wide range of sequence lengths. Additionally, the Lasso commitment time for CP-Link interoperability (excluded from the main table) scales from 107.5 ms at n=128 to 2,369.5 ms at n=1024 for BERT-Freivalds, reflecting the $O(n^2)$ growth of the attention matrix. In contrast, Nyström-Freivalds CP-Link commitment grows from 150.5 ms (n=128) to only 1,087.5 ms (n=4096), confirming that the low-rank kernel limits commitment cost as well as proving cost.

7.3.4 Combined Effect. Having decomposed each optimization axis in isolation, we now combine them to quantify the total reduction across the two extremes of our design space: standard BERT attention with naive in-circuit matrix multiplication—the conventional baseline—versus Nyström attention with Freivalds verification, our full scheme. Table 5 contrasts these two diagonal corners.

Table 5: Combined effect: BERT + Naive (baseline) vs. Nyström + Freivalds (full scheme) on Tiny (L=2). All metrics are per single layer (averaged over L layers). OOM entries (—) indicate memory exhaustion at 128 GB. Proof Size excludes CP-Link public instances.

	n=128		n=256		n=512		n=1024		n=2048		n=4096	
	BERT-N	Nys-F	BERT-N	Nys-F	BERT-N	Nys-F	BERT-N	Nys-F	BERT-N	Nys-F	BERT-N	Nys-F
Constr. (M)	4.78	0.54	19.07	0.96	—	1.79	—	3.47	—	6.83	—	13.54
Setup (s)	95.21	17.09	393.03	32.42	—	54.03	—	108.52	—	222.05	—	425.36
G16 Cmt (ms)	46.5	45.0	147.0	74.5	—	137.0	—	252.5	—	464.0	—	872.5
Lasso Cmt (ms)	98.0	189.0	203.0	233.5	—	280.0	—	405.0	—	614.0	—	945.5
Prove (s)	17.06	5.60	73.63	7.78	—	10.62	—	17.54	—	32.01	—	63.74
Verify (ms)	273.5	616.5	417.5	759.0	—	789.0	—	1,073.0	—	1,141.5	—	1,717.0
Proof Size (KB)	374.22	1,018.28	499.38	1,090.81	—	1,180.34	—	1,307.12	—	1,466.91	—	1,699.94
Peak RAM (GB)	21.28	3.70	79.63	7.24	113.16	11.12	113.04	18.81	113.09	34.63	113.18	69.02

At n=256, this combination reduces per-layer constraints from 19.07M (BERT-Naive) to 0.96M (Nyström-Freivalds)—a ∼19.9× reduction. The per-layer setup time shrinks from 393.03 s to 32.42 s (12.1×), and prove time drops from 73.63 s to 7.78 s (9.5×). Even at the shortest sequence (n=128), where both methods are feasible, the combined savings reach 8.9× in constraints, 5.6× in setup, and 3.0× in prove time. Peak memory drops from 21.28 GB to 3.70 GB (5.8×) at n=128 and from 79.63 GB to 7.24 GB (11.0×) at n=256. These joint savings show that combining low-rank attention (structural, $O(n)$ vs. $O(n^2)$) and Freivalds verification (constant-factor, ∼6×) makes it practical to generate zero-knowledge proofs of transformer attention even at sequence lengths that were difficult to reach under a limited memory budget. Full MLP cost breakdowns are deferred to Appendix A.

7.3.5 Multi-Spec Scaling. Figures 5 and 6 extend the two single-axis comparisons of Tables 3 and 4 from the Tiny configuration to the full BERT model family. The two figures decompose our contribution into its two constituent parts: the first isolates the benefit of Freivalds verification by holding the attention mechanism fixed (Nyström) and comparing naive vs. Freivalds matrix multiplication; the second isolates the benefit of low-rank attention by holding the verification method fixed (Freivalds) and comparing BERT vs. Nyström attention.

Both plots use log-log scale and report per-layer prove time (seconds) against sequence length n. Each line terminates at the largest n feasible within our 128 GB memory budget. In Figure 5, red lines correspond to Naive and blue lines to Freivalds; in Figure 6, orange lines correspond to BERT and green lines to Nyström . Within each color family, lighter shades represent smaller configurations and darker shades represent larger ones. Legend abbreviations: N/F = Naive/Freivalds; Bt/Ny = BERT/Nyström .

Freivalds extends the feasibility envelope across all configurations. As Figure 5 shows, Freivalds (blue) sits uniformly below Naive (red) by a 6.2–9.9× constant factor in per-layer prove time. This constant saving is decisive at long sequences: at every configuration the Naive baseline exhausts memory well before Freivalds, which extends the maximum feasible sequence length by 2–8×.

Figure 6 compares standard BERT attention (orange) against Nyström attention (green), with both methods proved using identical Freivalds-based matrix multiplication verification. The slope

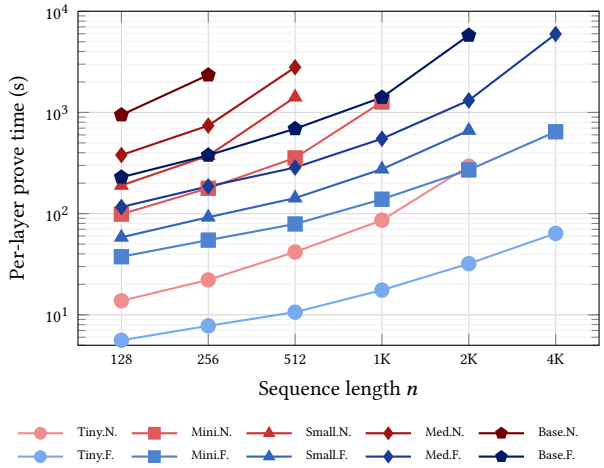


Figure 5: Effect of Freivalds verification on per-layer prove time: Nyström attention with Naive (red) vs. Freivalds (blue) matrix multiplication, as a function of sequence length n.

difference is the most striking feature: Nyström 's $O(n)$ curves (green) maintain slope ≈ 1, while BERT's $O(n^2)$ curves (orange) exhibit slope ≈ 2. The quadratic growth translates directly into memory exhaustion: at every configuration, BERT-Freivalds (orange) exhausts our 128 GB memory budget at a substantially shorter n than Nyström -Freivalds (green). The feasibility gap widens as d grows: at Tiny (d=128, lightest shades), BERT caps at n=1024 while Nyström reaches n=4096, a 4× extension; at Base (d=768, darkest shades), BERT caps at n=256 while Nyström reaches n=2048, an 8× extension.

7.4 CONCLUSION AND FUTURE WORK

Practical impact. Our measurements across all five configurations (Figures 5 and 6) demonstrate that the combination of Nyström low-rank attention and Freivalds randomized matrix-multiplication verification yields multiplicative constraint savings that translate directly into longer provable sequences.

Memory as the practical bottleneck. Failed runs terminate during the Groth16 trusted setup phase, which materializes the full

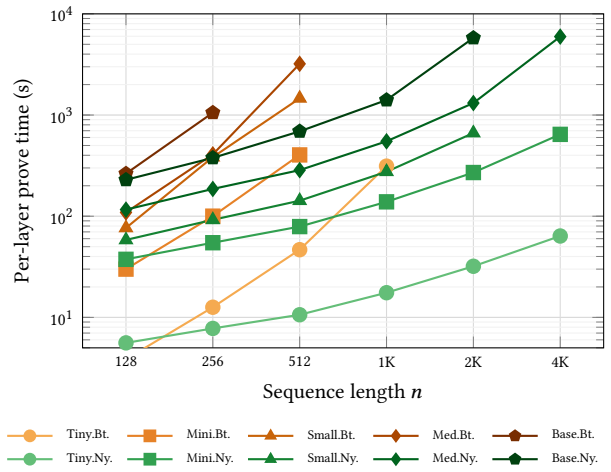


Figure 6: Effect of low-rank attention on per-layer prove time: BERT (orange) vs. Nyström (green) attention under Freivalds verification, as a function of sequence length n.

constraint system in RAM. Peak RSS for failed runs reaches ∼118–122 GB (the effective usable limit on our 128 GB machine), confirming that setup memory—not prover computation time—is the dominant scaling bottleneck.

The MLP frontier. While our work removes attention as the scaling bottleneck, the feed-forward MLP block emerges as the next memory frontier. We measured the cost of proving the MLP sub-circuit—identical between BERT and Nyströmformer and shared by both pipelines—and report the detailed results in Appendix A. The measurements confirm that, once attention is optimized, the MLP becomes the dominant constraint on the maximum provable sequence length: at d=512 and above, MLP exceeds 128 GB at all sequence lengths including n=128. Proving full-model transformer inference at sequence lengths beyond what this work demonstrates therefore requires either reducing the MLP's per-layer constraint footprint or infrastructure-level solutions such as streaming setup, distributed proving, or GPU-accelerated SNARK libraries. We view efficient ZK proving of the MLP block as a natural follow-up direction.

Per-layer cost is depth-invariant: implications for extrapolation. A consistent finding across our measurements is that per-layer attention circuit cost depends only on hidden dimension d and sequence length n, not on model depth L. Small (L=4, d=512) and Medium (L=8, d=512) yield identical per-layer constraints, setup, and prove time; Medium's full-model costs are exactly 2× Small's. This depth-invariance implies that our results extrapolate to deeper architectures through simple multiplication by L: a 24-layer model with Base-scale dimensions would require ∼2× the prove time and setup we report for L=12. The fundamental constraint is per-layer memory, not depth.

Limitations. Our work has several limitations. *(i) Single-machine CPU-only environment.* All measurements use an Apple M1 Ultra without GPU acceleration. GPU-accelerated proving or distributed proving across multiple machines would shift relative costs, particularly for larger circuits. *(ii) Accuracy not evaluated.* We do not measure the accuracy impact of the Nyström approximation, SigmoidGELU substitution, or 8-bit quantization, since circuit cost is independent of weight values. End-to-end task accuracy of a pretrained Nyströmformer with these modifications is a separate question deserving dedicated study.*(iii) Trusted setup.* Our Groth16-based system requires a circuit-specific trusted setup whose memory cost is our primary scaling bottleneck. Transparent SNARKs (e.g., FRI-based or IPA-based) would eliminate this at the cost of larger proof sizes and slower verification.*(iv) MLP sub-circuit.* The attention sub-circuit is made tractable by our techniques, but the MLP sub-circuit remains memory-bound at $d \geq 256$. Proving the full transformer (attention + MLP + LayerNorm) at the scales attempted here requires further techniques for the MLP block.

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. arXiv:2004.05150 [cs.CL] https://arxiv.org/abs/2004.05150

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs.CL] https://arxiv.org/abs/2005.14165

[3] Matteo Campanelli, Antonio Faonio, Dario Fiore, Anaïs Querol, and Hadrián Rodríguez. 2020. Lunar: a Toolbox for More Efficient Universal and Updatable zkSNARKs and Commit-and-Prove Extensions. Cryptology ePrint Archive, Paper 2020/1069. https://eprint.iacr.org/2020/1069

[4] Matteo Campanelli, Dario Fiore, and Anaïs Querol. 2019. LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs. Cryptology ePrint Archive, Paper 2019/142. https://doi.org/10.1145/3319535.3339820

[5] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) *(EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 560–574. https://doi.org/10.1145/3627703.3650088

[6] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating Long Sequences with Sparse Transformers. arXiv:1904.10509 [cs.LG] https://arxiv.org/abs/1904.10509

[7] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. 2022. Rethinking Attention with Performers. arXiv:2009.14794 [cs.LG] https://arxiv.org/abs/2009.14794

[8] Mingshu Cong, Tsz Hon Yuen, and Siu-Ming Yiu. 2024. zkMatrix: Batched Short Proof for Committed Matrix Multiplication. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security* (Singapore, Singapore) *(ASIA CCS '24)*. Association for Computing Machinery, New York, NY, USA, 289–305. https://doi.org/10.1145/3634737.3645003

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL] https://arxiv.org/abs/1810.04805

[10] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. 2021. ZEN: An Optimizing Compiler for Verifiable, Zero-Knowledge Neural Network Inferences. Cryptology ePrint Archive, Paper 2021/087. https://eprint.iacr.org/2021/087

[11] Rusins Freivalds. 1977. Probabilistic Machines Can Use Less Running Time. In *IFIP Congress*. https://api.semanticscholar.org/CorpusID:45405368

[12] Zahra Ghodsi, Tianyu Gu, and Siddharth Garg. 2017. SafetyNets: Verifiable Execution of Deep Neural Networks on an Untrusted Cloud. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/6048ff4e8cb07aa60b6777b6f7384d52-Paper.pdf

[13] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian Error Linear Units (GELUs). arXiv:1606.08415 [cs.LG] https://arxiv.org/abs/1606.08415

[14] Byeongjun Jang, Gweonho Jeong, Hyuktae Kwon, Hyunok Oh, and Jihye Kim. 2024. VECTIS: Efficient Batching Framework for Group-based CP-SNARKs. Cryptology ePrint Archive, Paper 2024/1405. https://eprint.iacr.org/2024/1405
[15] Ivan Korec and Jiří Wiedermann. 2014. Deterministic Verification of Integer Matrix Multiplication in Quadratic Time. In *SOFSEM 2014: Theory and Practice of Computer Science*, Viliam Geffert, Bart Preneel, Branislav Rovan, Július Štuller, and A. Min Tjoa (Eds.). Springer International Publishing, Cham, 375–382.
[16] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vCNN: Verifiable Convolutional Neural Network Based on zk-SNARKs. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4254–4270. https://doi.org/10.1109/TDSC.2023.3348760
[17] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) *(CCS '21)*. Association for Computing Machinery, New York, NY, USA, 2968–2985. https://doi.org/10.1145/3460120.3485379
[18] Meta. 2024. Llama 4 Model Card. GitHub repository documentation. https://github.com/meta-llama/llama-models/blob/main/models/llama4/MODEL_CARD.md Accessed: 2025-12-31.
[19] OpenAI. 2025. GPT-5 Model. OpenAI API Documentation. https://platform.openai.com/docs/models/gpt-5 Accessed: 2025-12-31.
[20] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. 2019. Compressive Transformers for Long-Range Sequence Modelling. arXiv:1911.05507 [cs.LG] https://arxiv.org/abs/1911.05507
[21] Srinath Setty, Justin Thaler, and Riad Wahby. 2023. Unlocking the lookup singularity with Lasso. Cryptology ePrint Archive, Paper 2023/1216. https://eprint.iacr.org/2023/1216
[22] Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkLLM: Zero Knowledge Proofs for Large Language Models. arXiv:2404.16109 [cs.LG] https://arxiv.org/abs/2404.16109
[23] Justin Thaler. 2013. Time-Optimal Interactive Proofs for Circuit Evaluation. In *Advances in Cryptology – CRYPTO 2013*, Ran Canetti and Juan A. Garay (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 71–89.
[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. arXiv:1706.03762 [cs.CL] https://arxiv.org/abs/1706.03762
[25] Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-Attention with Linear Complexity. arXiv:2006.04768 [cs.LG] https://arxiv.org/abs/2006.04768
[26] Jiasi Weng, Jian Weng, Gui Tang, Anjia Yang, Ming Li, and Jia-Nan Liu. 2023. pvCNN: Privacy-Preserving and Verifiable Convolutional Neural Network Testing. arXiv:2201.09186 [cs.CR] https://arxiv.org/abs/2201.09186
[27] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 38–45.
[28] Jeffrey T. H. Wong, Cheng Zhang, Xinye Cao, Pedro Gimenes, George A. Constantinides, Wayne Luk, and Yiren Zhao. 2025. A3 : an Analytical Low-Rank Approximation Framework for Attention. arXiv:2505.12942 [cs.CL] https://arxiv.org/abs/2505.12942
[29] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. 2021. Nyströmformer: A Nyström-Based Algorithm for Approximating Self-Attention. arXiv:2102.03902 [cs.CL] https://arxiv.org/abs/2102.03902
[30] Yancheng Zhang, Mengxin Zheng, Xun Chen, Jingtong Hu, Weidong Shi, Lei Ju, Yan Solihin, and Qian Lou. 2025. zkVC: Fast Zero-Knowledge Proof for Private and Verifiable Computing. arXiv:2504.12217 [cs.CR] https://arxiv.org/abs/2504.12217

A MLP Circuit Costs

A complete transformer layer consists of two sub-circuits: attention and the feed-forward MLP block. While the main text focuses on the attention sub-circuit—the target of our zkDietFormer optimizations—we additionally measured the cost of proving the MLP sub-circuit, whose constraint count is the same for BERT and Nyströmformer because the MLP dimensions depend only on n and d, not on the attention mechanism. Table 6 reports per-layer average MLP costs for the Tiny configuration (L=2); all timing and memory figures are the mean of the BERT and Nyström runs. These measurements confirm that, once attention is optimized via Nyström + Freivalds, the MLP block becomes the dominant cost of full-model proving. At n=1024, per-layer MLP proving takes 489.46 s versus 17.54 s for Nyström -Freivalds attention—a 27.9× ratio. The MLP block is also the memory bottleneck of the full transformer: it consumes 113.17 GB peak RAM at n=1024 and exceeds 128 GB at $n \geq 2048$, whereas Nyström + Freivalds attention remains feasible up to n=4096 at 69.02 GB peak memory.

For reference, proving a complete Tiny transformer layer (attention + MLP) at n=1024 requires approximately 970.85 s of per-layer setup and 507.00 s of per-layer proving (~25 min total per layer). We report these measurements primarily as a baseline for future work on scalable ZK proving of the MLP block, which we identify as the next frontier following our resolution of the attention bottleneck.

Table 6: MLP + LayerNorm costs for the Tiny configuration (L=2, d=128). The MLP circuit is architecture-independent, so the constraint count is the same for BERT and Nyström ; all timing and memory figures are the mean of the two runs. All metrics are per single layer (averaged over L layers). OOM at $n \geq 2048$. Proof Size excludes CP-Link public instances.

			Commit (ms)					
n	Constr. (M)	Setup (s)	G16	Lasso	Prove (s)	Verify (ms)	Proof Size (KB)	Peak RAM (GB)
128	0.98	92.48	120.8	138.0	27.00	369.3	520.59	26.55
256	1.96	185.85	232.8	186.0	56.57	389.3	572.72	53.22
512	3.92	381.66	461.0	293.5	129.67	576.1	654.53	102.25
1024	7.84	862.33	1,157.8	449.8	489.46	633.5	746.03	113.17
2048	—	—	—	—	—	—	—	114.09

$n \geq 2048$: OOM during setup (peak RSS ≈114 GB).